Departament d'Enginyeria Informàtica i Matemàtiques
[DEIM] Universitat Rovira i Virgili

Arquitectura de Computadors
AC
Curso 26/27
Primera Convocatòria
Pràctica 2: Comportament Predictors de Salt *Simplescalar*

Simulació processador Superescalar:

Predictors de Salt

La pràctica consisteix en l'anàlisi de l'estructura i el comportament de diferents predictors de salt en un processador Superescalar amb execució fora d'ordre (OoO). Per fer-ho, s'utilitzarà el simulador Simplescalar i s'avaluarà el comportament del processador executant els programes de prova SpecCPU2000.

La predicció de salts permet en un processador superescalar l'execució especulativa de les instruccions d'una de les dues alternatives d'un salt (instrucció de salt: branch). Així, el processador davant d'un hazard (risc) de control, pot aprofitar els cicles que perdria fins a saber la resolució del salt, en provar una de les dues alternatives i guanyar d'aquesta manera aquests cicles d'espera.

Direm que un predictor és bo quan encerta el camí correcte amb una alta probabilitat. Amb freqüència els predictors bons són complexos i utilitzen estructures de dades per emmagatzemar el comportament de les instruccions de salt. Un Branch Address/Target Buffer (BTB) emmagatzema les adreces efectives de les instruccions de salt a mesura que les va executant. Un Return Address Stack (RAS) emmagatzema en forma de pila les adreces de retorn de les instruccions Call. De la mateixa manera, un Branch History Register (BHR) emmagatzema la resolució (Taken / Not_Taken) dels darrers salts acumulats. Pot ser Global (GBHR) o associat a cada instrucció de salt de manera individual Per Address BHR (PaBHR). Per acabar, un Pattern History Table (PHT) emmagatzema la predicció que es farà en un possible salt. D'aquesta manera, manté un comptador saturat de 2 bits i utilitza el bit de més pes per a la predicció. Aquesta estructura pot ser global, individual a cada salt o fins i tot compartir-se entre alguns salts que tinguin comportaments semblants.

Simplescalar, mitjançant els simuladors sim-bpred i sim-outorder, permet configurar cadascuna d'aquestes quatre estructures per analitzar el comportament de diferents configuracions.

Per a aquesta pràctica, es farà servir un subconjunt de 5 benchmarks disponibles a través de la imatge i també de l'espai moodle de l'assignatura.

Comentaris

- El grup de pràctiques estarà format per 3 membres: **2 PERSONES** i ChatGPT (o equivalent :-)**. Si el feu servir**, el text generat per l'eina constarà a l'informe en lletra ***cursiva***.
- Es realitzarà una entrevista presencial amb els integrants del grup (humans) a la sessió de laboratori que tenen assignada. On es defensarà amb arguments la feina realitzada a qualsevol part de la pràctica.
- L'informe (obligatòriament en PDF), el vídeo resum de la pràctica i els fitxers auxiliars que s'hagin utilitzats per les diferents eines/simuladors es guardaran al moodle abans de fer l'entrevista (units en format ZIP).

Especificació

La tasca d'aquesta pràctica es divideix en dues fases: (1) Estudi dels predictors que es poden simular amb el Simplescalar i (2) Modificació del Simplescalar per simular un nou predictor de salts.

1a Fase:

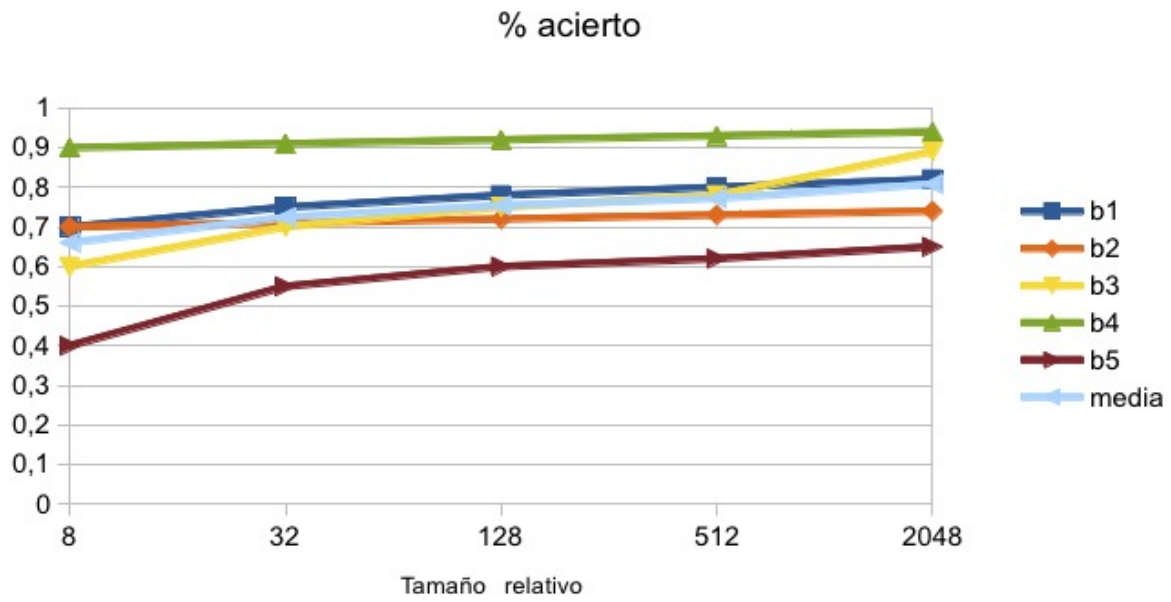
Els predictors que implementa Simplescalar son: nottaken, taken, perfect, bimodal, 2-level y comb.

Es faran servir per avaluar el comportament quant a IPC i percentatge d'encerts de les següents alternatives de predictors de salts:

- **nottaken**: opció: -bpred nottaken
- **taken**: opció: -bpred taken
- **perfect**: opció: -bpred perfect
- **bimodal**: opció: -bpred bimod
  - Mida del PHT <l2-size> : 8,32,128,512,2048
  - -bpred:bimod X
- **Gshare**: opción: -bpred 2lev
  - Mida del BHR <l1-size>: 1 i del PHT<l2-size>: 8,32,128,512,2048
  - -bpred:2lev 1 <X> <$\log_2$X> 1
- **Gag (Gselect):** opció: -bpred 2lev
  - Mida del BHR <l1-size>: 1 i del PHT<l2-size>: 8,32,128,512,2048
  - -bpred:2lev 1 <X> <$\log_2$X> 0
- **Pag**: opció: -bpred 2lev
  - Mida del BHR <l1-size> i del PHT<l2-size> respectivament: (4-4), (8-16), (16-64), (32-256), (64-1024), (32-2048) son (Y-X)
  - -bpred:2lev <Y> <X> <$\log_2$X> 0

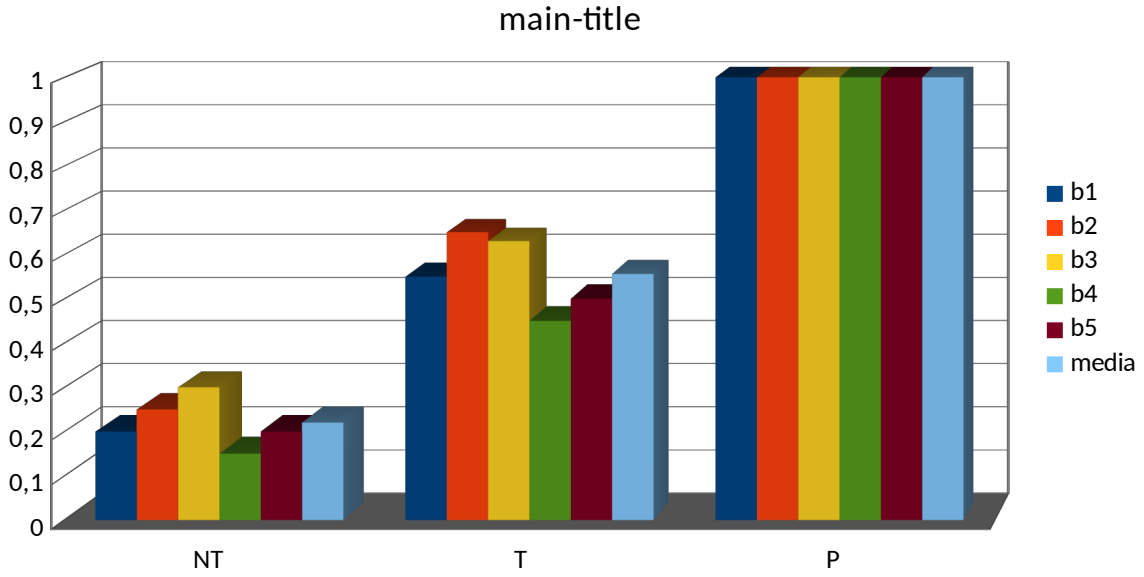
L'objectiu és observar i generar gràfiques que mostrin el comportament de l'IPC i el percentatge d'encert (.bpred_dir_rate) per a diferents valors de configuració dels predictors amb els cinc benchmarks que es disposen de la pràctica anterior.

Per als diferents estudis que s'han de realitzar, i si no s'indica el contrari, tingueu en compte els comentaris següents:



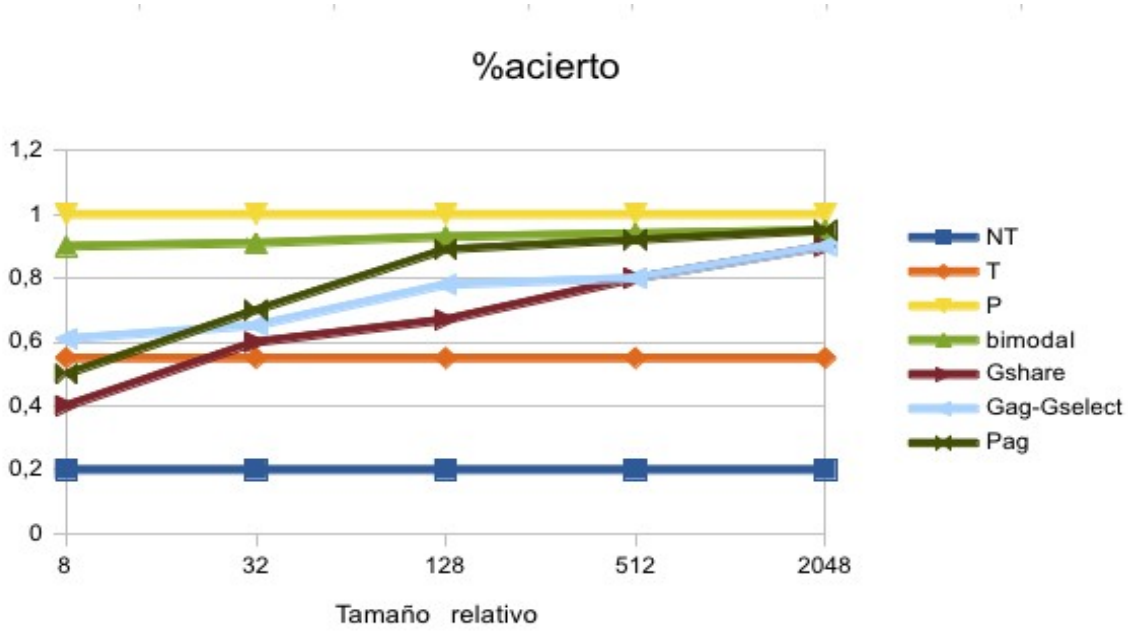
1) Es mostrarà una gràfica de cada paràmetre per a cadascun dels set predictors indicats anteriorment, on a l'eix de les Y hi haurà el paràmetre i a l'eix de les X les diferents configuracions en mida del predictor estudiat. A la mateixa gràfica apareixeran els cinc benchmarks i la mitjana dels mateixos.

2) Per als predictors estàtics es poden resumir tots en una única gràfica de la manera següent:



3) Per a cada estudi es presentaran gràfiques amb els resultats individuals de cada benchmark i amb la mitjana de tots.

4) També es generaran gràfiques resumeixen del comportament de cadascun dels predictors. A la mateixa gràfica apareixeran els set predictors tenint a l'eix de les X les diferents configuracions.



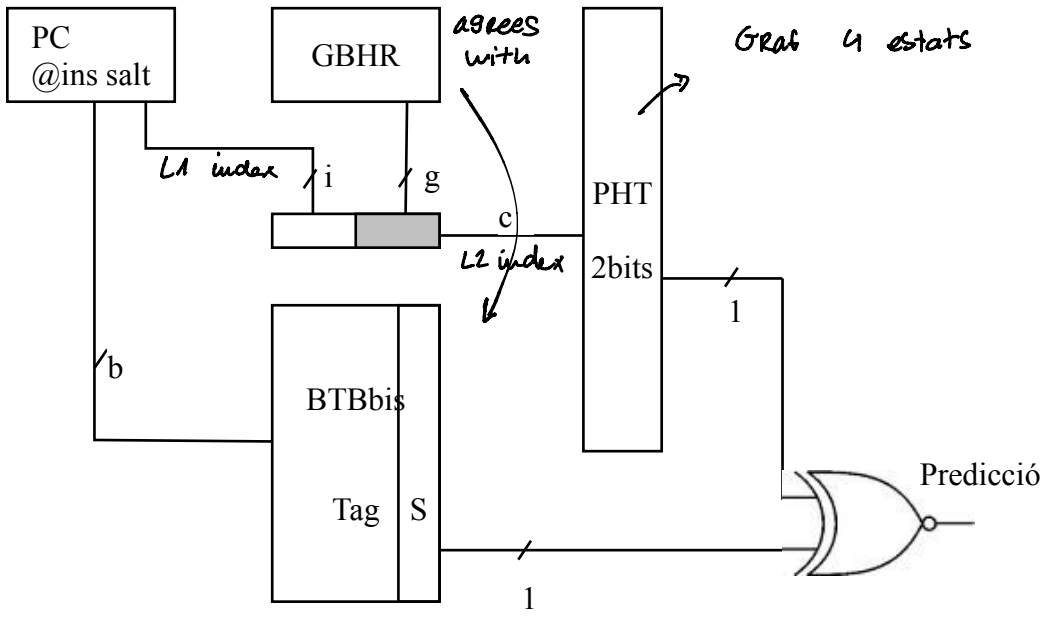
5) Per a cada benchmark simulat, se saltaran 100 milions d'instruccions i es recol·lectaran les estadístiques per als 100 milions següents. A més, es faran servir les dades d'entrada REF.

6) La mida de BTB i RAS no es modificaran del valor per defecte.

7) Per a la resta de paràmetres del simulador s'utilitzaran els valors per defecte, a excepció del bus d'accés a memòria (de 32 bytes) i de la seva latència (300 cicles inicials i 2 per accés consecutiu).

2 fase:

Implementació del predictor de salts AGREE.



El predictor manté la informació dels salts executats en quatre estructures:

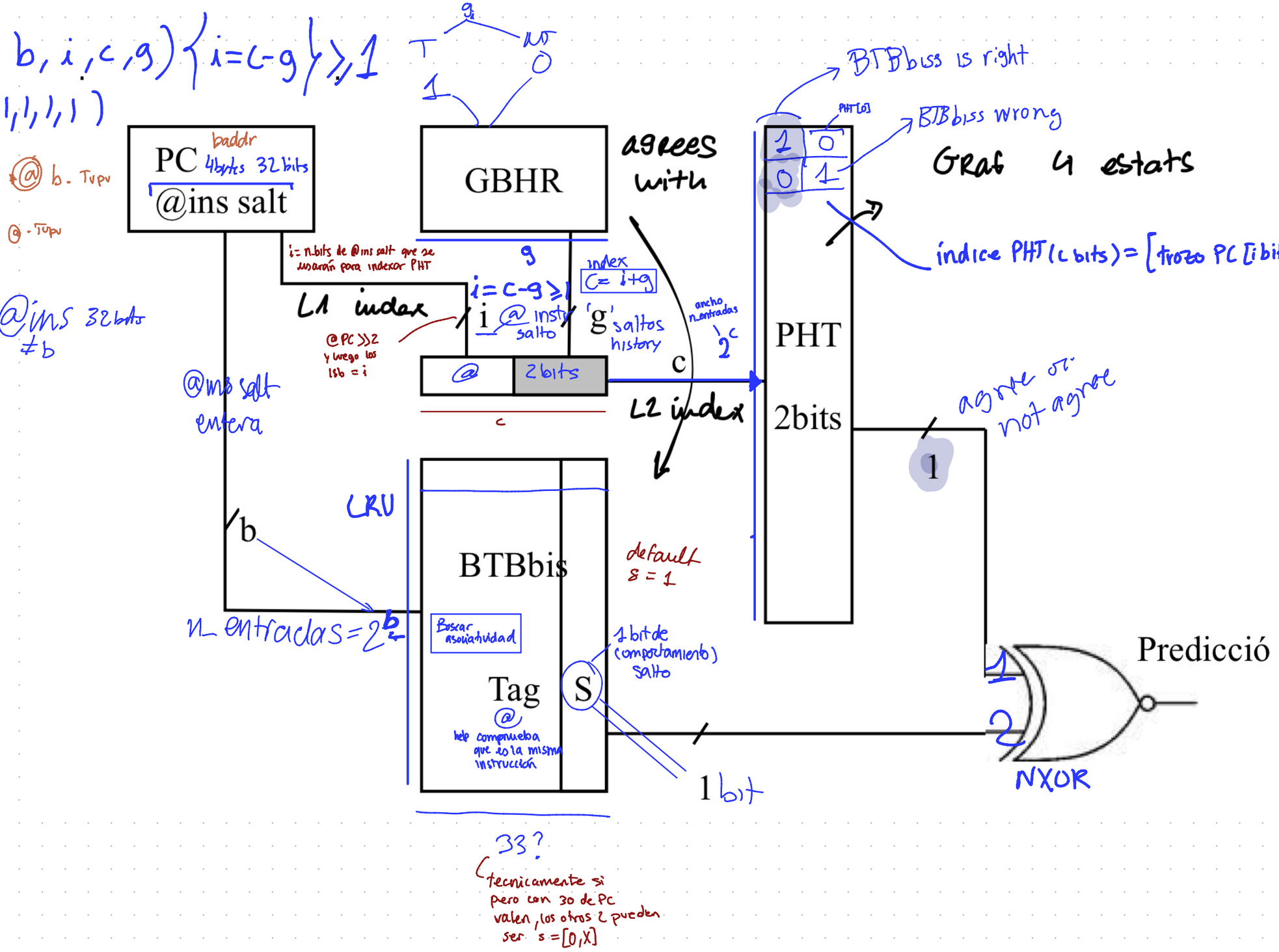
1. Global Branch History Register (GBHR): emmagatzema l'historial dels últims 'g' salts que s'han executat al processador. Guarda un 1 si el salt ha estat Taken i un 0 si ha estat NotTaken. (Internament serà un registre de 32 bits del qual utilitzarem els 'g' bits menys significatius).

2. Branch Target Buffer (BTBbis): és una taula de dimensió 2^b entrades, cadascuna d'elles formada per una etiqueta (Tag) i un bit de comportament del salt. S'hi accedeix de manera completament associativa a partir de l'adreça de la instrucció de salt. L'objectiu d'aquesta taula és emmagatzemar el comportament de la instrucció de salt. El Tag garanteix que es tracta de la mateixa instrucció i el bit indica Taken (1) o NotTaken (0). De fet, la taula s'omple amb el comportament observat la primera vegada que s'executa el salt. Es podria utilitzar el BTB ja implementat a sim-outorder, però per evitar complicar encara més el codi és preferible crear un BTB propi (d'aquí el nom BTBbis).

3. Pattern History Table (PHT): és una taula on s'emmagatzemen dos bits per implementar un comptador saturat del comportament dels salts. Es modifica lleugerament el significat d'aquests dos bits, de manera que cada vegada que el comportament real del salt coincideix amb el valor emmagatzemat al BTBbis el comptador s'incrementa, i cada vegada que discrepa es decrementa. Així s'obtenen quatre estats, dos dels quals (els que tenen el bit més significatiu a '1') indiquen que el bit del BTBbis té raó, mentre que els altres dos (els que tenen el bit més significatiu a '0') indiquen que el bit del BTBbis no té raó. S'hi accedeix de manera directa a partir

$V=(b,i,c,g)\{i=c-g\}\geq 1$

$V_1=(1,1,1,1)$



@ - Tupv

@ b. Tupv

@ins 32bits ≠b

33?

(tecnicamente si pero con 30 de PC valen, los otros 2 pueden ser s=[0,X]

de l'índex obtingut de la concatenació del contingut del GBHR (g bits) i de bits de l'adreça de la instrucció de salt (i bits).

4. Obtenció de la predicció: la predicció s'obté a partir de l'operació NXOR entre el bit més significatiu de l'entrada corresponent de la PHT i el bit del BTBbis.

   Així: (PHT_1,BTBbis) → Predicció

   - (00) → 1 indica «predicció incorrecta: NotTaken → Taken».
   - (01) → 0 indica «predicció incorrecta: Taken → NotTaken».
   - (10) → 0 indica «predicció correcta: NotTaken → NotTaken».
   - (11) → 1 indica «predicció correcta: Taken → Taken».

Cada vegada que s'executa un salt, aquestes estructures s'accedeixen dues vegades. La primera per obtenir una predicció i actuar en conseqüència dins del pipeline, i la segona per actualitzar el comportament real del salt executat i refinar les prediccions futures.

La predicció s'obté a partir de la funció NXOR entre el bit més significatiu de l'entrada seleccionada de la PHT i el bit de comportament (S) del BTBbis. Cal tenir en compte que el BTBbis és completament associatiu i, per tant, serà necessari accedir a totes les seves entrades per comprovar si es produeix un HIT o un MISS.

L'actualització es realitza sempre sobre el GBHR i la PHT principal. El BTBbis només s'actualitza si s'hi ha produït un MISS; en aquest cas s'hi afegeix la nova instrucció de salt juntament amb el primer comportament observat. En ser una estructura completament associativa, necessita un algorisme de reemplaçament, que serà LRU (Least Recently Used).

Els paràmetres que defineixen aquest predictor són:

1. Nombre de bits del GBHR → defineix 'g'.
2. Nombre d'entrades del BTBbis → defineix 'b'.
3. Nombre d'entrades de la PHT → defineix 'c'.
4. A partir d'aquests valors es dedueix 'i' = 'c' - 'g' (que no pot ser inferior a 1).

Els valors inicials de totes aquestes estructures són bits a '1'. D'aquesta manera, la predicció inicial de qualsevol salt serà Taken.

Per modificar SimpleScalar i afegir un nou predictor, es pot prendre com a referència la implementació d'algun predictor ja existent i duplicar-hi o afegir-hi el codi necessari. En aquest cas, el predictor **2lev** és prou semblant i servirà de model.

Les parts de SimpleScalar que cal modificar són:

A bpred.h

- Afegir el nou predictor a la llista de l'enumeració bpred_class.

- En la definició dels tipus de predictors, afegir a l'estructura struct bpred_dir_t, dins de la subestructura two, les noves variables i punters necessaris. Tingueu en compte que el nivell 1 (level-1) podria correspondre al BTBbis i el nivell 2 (level-2) a la PHT.

A sim-outorder.c

- Registrar els paràmetres de configuració del predictor dins de sim_reg_options, mitjançant una crida similar a:

  ```
  opt_reg_int_list(opt,"-bpred:agree", ...)
  ```

- Definir les variables predictor_config i predictor_nelt.
- Afegir a sim_check_options la lectura dels paràmetres del predictor i la corresponent crida a la funció de creació del predictor:

  ```
  bpred_create(...)
  ```

  Es recomana prendre com a referència la implementació del predictor 2lev.

A bpred.c

- Afegir a bpred_create i bpred_dir_create un nou cas (case) d'inicialització del predictor, on es reservi la memòria necessària i s'inicialitzin les variables i taules a partir dels paràmetres especificats.
- Afegir a bpred_config i bpred_dir_config un nou cas per mostrar la configuració del predictor a la sortida del simulador.
- Afegir a bpred_reg_stats un nou cas per poder visualitzar els resultats de la simulació.
- Afegir a bpred_lookup i bpred_dir_lookup un nou cas per obtenir l'adreça de la següent instrucció a partir de la predicció realitzada sobre la instrucció de salt que s'acaba de llegir (fetch), sempre que aquesta es trobi al BTB.

  Aclariment: bpred_dir_lookup retorna l'adreça de l'entrada corresponent de la taula PHT (L2 Table), mentre que bpred_lookup la desa al camp pdir1 de la pròpia instrucció perquè, en una posterior crida a bpred_update, ja es disposi de la posició calculada dins la PHT. (Es tracta d'una optimització).

- Afegir a bpred_update el cas necessari per actualitzar el GBHR, el BTBbis (L1 Table) i la PHT (L2 Table) segons correspongui. Cal tenir en compte que aquestes dues darreres taules es poden actualitzar utilitzant les adreces calculades anteriorment (l'optimització esmentada).

En general, la simulació a SimpleScalar s'inicia a main.c. De les funcions esmentades anteriorment, primer es crida sim_reg_options(), després sim_check_options() i finalment sim_main(). Les tres estan implementades dins del codi principal del simulador sim-outorder, i és a partir d'aquest moment que s'inicia la simulació pròpiament dita.

La funció sim_main() implementa el pipeline del processador superescalar. Per fer-ho, crida diferents funcions que simulen el comportament del processador; entre elles ens interessa especialment la funció ruu_fetch().

ruu_fetch() s'encarrega de llegir instruccions de memòria, simular la memòria cache, el TLB i el predictor de salts. En llegir una instrucció, la descodifica parcialment i, si detecta que es tracta d'una instrucció de salt, crida bpred_lookup() perquè retorni l'adreça de la següent instrucció que, segons la predicció realitzada, s'haurà de continuar buscant a memòria.

La funció bpred_update() actualitza la informació del predictor a partir de l'execució real de la instrucció de salt. Generalment és cridada des de ruu_commit(), però també pot ser invocada abans des de ruu_writeback() o fins i tot des de ruu_dispatch().

Un cop implementat el predictor, s'afegirà el seu comportament a les gràfiques obtingudes en les pràctiques anteriors.

Paràmetres a analitzar del nou predictor

- **Agree**: opció -bpred agree
    - Mida del BTBbis <l1-size> i de la PHT <l2-size> respectivament:

      (4-8), (8-32), (16-128), (32-512), (64-2048) corresponents a (Y-X).
    - Amplada del GBHR <g>: 2, 3, 4, 6 i 8.
    - Així, la configuració de c = i + g queda:
        - (1 + 2 → 3)
        - (2 + 3 → 5)
        - (3 + 4 → 7)
        - (3 + 6 → 9)
        - (3 + 8 → 11)
    - Sintaxi:

      `-bpred:agree <Y> <X> <g> 0`

Tasques finals

1. Comenteu a quin dels predictors Gag, Pag, Gap o Pap s'assembla més aquest predictor i amb quins paràmetres.
2. Un cop identificat el predictor ja existent a SimpleScalar que més s'assembla al predictor implementat, simuleu tots dos predictors i compareu-ne els resultats.

La funció sim_main() implementa el pipeline del processador superescalar. Per fer-ho, crida diferents funcions que simulen el comportament del processador; entre elles ens interessa especialment la funció ruu_fetch().

ruu_fetch() s'encarrega de llegir instruccions de memòria, simular la memòria cache, el TLB i el predictor de salts. En llegir una instrucció, la descodifica parcialment i, si detecta que es tracta d'una instrucció de salt, crida bpred_lookup() perquè retorni l'adreça de la següent instrucció que, segons la predicció realitzada, s'haurà de continuar buscant a memòria.

La funció bpred_update() actualitza la informació del predictor a partir de l'execució real de la instrucció de salt. Generalment és cridada des de ruu_commit(), però també pot ser invocada abans des de ruu_writeback() o fins i tot des de ruu_dispatch().

Un cop implementat el predictor, s'afegirà el seu comportament a les gràfiques obtingudes en les pràctiques anteriors.

Paràmetres a analitzar del nou predictor

- **Agree**: opció -bpred agree
    - Mida del BTBbis <l1-size> i de la PHT <l2-size> respectivament:

      (4-8), (8-32), (16-128), (32-512), (64-2048) corresponents a (Y-X).
    - Amplada del GBHR <g>: 2, 3, 4, 6 i 8.
    - Així, la configuració de c = i + g queda:
        - (1 + 2 → 3)
        - (2 + 3 → 5)
        - (3 + 4 → 7)
        - (3 + 6 → 9)
        - (3 + 8 → 11)
    - Sintaxi:

      `-bpred:agree <Y> <X> <g> 0`

Tasques finals

1. Comenteu a quin dels predictors Gag, Pag, Gap o Pap s'assembla més aquest predictor i amb quins paràmetres.
2. Un cop identificat el predictor ja existent a SimpleScalar que més s'assembla al predictor implementat, simuleu tots dos predictors i compareu-ne els resultats.